

Property	Value
Outer side length	70mm
Turns per coil	25
Track width	0.5mm
Track spacing	0.25mm
PCB thickness	1.6mm

Table 1: Specification of PCB emitter coils in Anser field generator [?]

The Anser electromagnetic tracking (EMT) system infers the position and orientation (pose) of a small sensor coil from eight measurements induced by a planar array of emitter coils. This is an inverse problem: the physics maps from pose to measurement, but the system must run this map backwards, recovering five pose coordinates from eight measured values.

This chapter builds the machinery needed to both understand and solve the problem. We first describe the physical system, then derive a forward map from the underlying physics. We then look at the characteristics of this map and ask when it can be inverted, at least locally, and turn to methods that can do this in practice.

The first class of methods we look at are the iterative methods, whose success depends on where they start. The second class are neural networks (NNs), which learn an approximate inverse directly. The chapter closes by introducing a hybrid approach, which combines the strengths of both.

0.1 The Anser System

Anser EMT [?] is the first fully open-source electromagnetic tracking (EMT) platform for image-guided interventions. It uses magnetic fields to determine the position and orientation of a sensor embedded in the field. It has two main components: a planar field generator, and a tracking sensor coil.

0.1.1 The Field Generator

The Anser EMT system uses a planar field generator consisting of 8 coils being driven at different frequencies. Each coil consists of a wound rectangular spiraling filament with specifications shown in table 1.

Eight of these coils are positioned on the PCB, as seen in Figure ??

[MISSING FIGURE]
FIGURE OF SINGLE COIL

Figure 1: FIGURE OF SINGLE COIL

[MISSING FIGURE]
Figure of 8 coil generator

Figure 2: Figure of 8 coil generator

0.1.2 The Tracking Sensor Coil

The tracking sensor records the flux, and due to the emitter coils running at different frequencies, is able to recover the 8 flux values due to each of the emitter coils after demodulation. These 8 fluxes are fed into the pose solving algorithm to determine the position and orientation of the sensor.

0.2 Magnetic Field Derivation

Each spiral coil is composed of a sequence of straight line filaments and Maxwell's equations in free space are linear [?, p. 59] so the total magnetic field will be the sum of the fields of each individual straight line filament. Our main goal then, is to find the magnetic field generated at an observer point p due to the current flowing along one of these straight line filaments, **a**.

We start by defining vectors **a**, **b**, and **c**.

- **a** points along the current filament.
- **b** points from the end of the filament to the observer.
- **c** points from the start of the filament to the observer.

Note that throughout I use the convention $|\mathbf{a}| = a$.

Now, we use the Biot Savart Law to find the magnetic field in each of the lines.

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \int \frac{d\mathbf{l} \times \mathbf{r}}{r^3} \quad (1)$$

To evaluate this integral we first parametrise $\mathbf{r}$ in a parameter t , yielding:

$$\begin{aligned} \mathbf{r}(t) &= \mathbf{c} - \mathbf{a}t, \quad t \in [0, 1] \\ d\mathbf{l} &= \mathbf{a}dt \end{aligned}$$

Substituting in gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \int_0^1 \frac{(\mathbf{a}dt) \times (\mathbf{c} - \mathbf{a}t)}{|\mathbf{c} - t\mathbf{a}|^3} \quad (2)$$

Noting that $\mathbf{a} \times \mathbf{a} = 0$, and that $\mathbf{a}$ and $\mathbf{c}$ are constants, this simplifies to:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} (\mathbf{a} \times \mathbf{c}) \int_0^1 \frac{dt}{|\mathbf{c} - t\mathbf{a}|^3} \quad (3)$$

The denominator must now be simplified.

$$|\mathbf{c} - t\mathbf{a}|^3 = (c^2 + t^2 a^2 - 2t(\mathbf{a} \cdot \mathbf{c}))^{3/2} \quad (4)$$

For simplicity, let $\beta = -(\mathbf{a} \cdot \mathbf{c})$

$$|\mathbf{c} - t\mathbf{a}|^3 = (a^2 t^2 + 2\beta t + c^2)^{3/2} \quad (5)$$

Completing the square:

$$|\mathbf{c} - t\mathbf{a}|^3 = a^3 \left(\left(t + \frac{\beta}{a^2} \right)^2 - \frac{\beta^2}{a^4} + \frac{c^2}{a^2} \right)^{3/2} \quad (6)$$

Now let $u = t + \frac{\beta}{a^2}$, $du = dt$, $u(0) = \frac{\beta}{a^2}$, $u(1) = 1 + \frac{\beta}{a^2}$. Also, let $\delta = \frac{c^2}{a^2} - \frac{\beta^2}{a^4}$. Subbing back into our Biot Savart Formula gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3} \int_{\frac{\beta}{a^2}}^{1+\frac{\beta}{a^2}} \frac{du}{(u^2 + \delta)^{3/2}} \quad (7)$$

We want to use the identity $\sec^2 \phi = 1 + \tan^2 \phi$, to do so, factor out $\delta^{3/2}$

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta^{3/2}} \int_{\frac{\beta}{a^2}}^{1+\frac{\beta}{a^2}} \frac{du}{\left(\frac{u^2}{\delta} + 1 \right)^{3/2}} \quad (8)$$

Now that it's in the correct form let: $\tan^2 \phi = \frac{u^2}{\delta}$, then:

$$\begin{aligned}\tan \phi &= \frac{u}{\sqrt{\delta}} \\ \sec^2 \phi d\phi &= \frac{1}{\sqrt{\delta}} du \\ \phi \left(\frac{\beta}{a^2} \right) &= \arctan \left(\frac{\beta}{a^2 \sqrt{\delta}} \right) \\ \phi \left(1 + \frac{\beta}{a^2} \right) &= \arctan \left(\frac{1 + \frac{\beta}{a^2}}{\sqrt{\delta}} \right)\end{aligned}$$

Substituting back in:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta^{3/2}} \int_{\arctan\left(\frac{\beta}{a^2 \sqrt{\delta}}\right)}^{\arctan\left(\frac{1}{\sqrt{\delta}}\left(1 + \frac{\beta}{a^2}\right)\right)} \frac{\sqrt{\delta} \sec^2 \phi d\phi}{(1 + \tan^2 \phi)^{3/2}} \quad (9)$$

Using the identities $\sec^2 \phi = 1 + \tan^2 \phi$ and $\cos \phi = \frac{1}{\sec \phi}$ and simplifying yields:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \int_{\arctan\left(\frac{\beta}{a^2 \sqrt{\delta}}\right)}^{\arctan\left(\frac{1}{\sqrt{\delta}}\left(1 + \frac{\beta}{a^2}\right)\right)} \cos \phi d\phi \quad (10)$$

Solving:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \left(\sin \left(\arctan \left(\frac{1}{\sqrt{\delta}} \left(1 + \frac{\beta}{a^2} \right) \right) \right) - \sin \left(\arctan \left(\frac{\beta}{a^2 \sqrt{\delta}} \right) \right) \right) \quad (11)$$

Now, we must consider a right angled triangle to work this out. $\sin \phi = \frac{o}{h}$, $\tan \phi = \frac{o}{a}$ therefore $\sin \left(\arctan \left(\frac{o}{a} \right) \right) = \frac{o}{\sqrt{o^2 + a^2}}$ by pythagoras.

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^3 \delta} \left(\left(\frac{1 + \frac{\beta}{a^2}}{\sqrt{\delta + \left(1 + \frac{\beta}{a^2}\right)^2}} \right) - \left(\frac{\beta}{\sqrt{\beta^2 + a^4 \delta}} \right) \right) \quad (12)$$

Recalling $\delta = \frac{c^2}{a^2} - \frac{\beta^2}{a^4}$, $\beta = -\mathbf{a} \cdot \mathbf{c}$, and looking at each of the denominators:

$$\begin{aligned}
\sqrt{\delta + \left(1 + \frac{\beta}{a^2}\right)^2} &= \sqrt{\frac{c^2}{a^2} - \frac{\beta^2}{a^4} + 1 + 2\frac{\beta}{a^2} + \frac{\beta^2}{a^4}} \\
&= \sqrt{1 + \frac{c^2}{a^2} - \frac{2}{a^2}(\mathbf{a} \cdot \mathbf{c})} \\
&= \frac{1}{a} \sqrt{a^2 + c^2 - 2(\mathbf{a} \cdot \mathbf{c})} \\
&= \frac{1}{a} |\mathbf{a} - \mathbf{c}| \\
&= \frac{b}{a}
\end{aligned}$$

$$\begin{aligned}
\sqrt{\beta^2 + a^4 \delta} &= \sqrt{\beta^2 + a^2 c^2 - \beta^2} \\
&= \sqrt{a^2 c^2} \\
&= ac
\end{aligned}$$

Subbing into our formula for $\mathbf{B}$ now gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{ac^2 - \frac{(\mathbf{a} \cdot \mathbf{c})^2}{a}} \left(\frac{1 - \frac{\mathbf{a} \cdot \mathbf{c}}{a^2}}{\frac{b}{a}} + \frac{\mathbf{a} \cdot \mathbf{c}}{ac} \right) \quad (13)$$

Simplifying a small bit gives:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{a^2 c^2 - (\mathbf{a} \cdot \mathbf{c})^2} \left(\frac{a^2 - \mathbf{a} \cdot \mathbf{c}}{b} + \frac{\mathbf{a} \cdot \mathbf{c}}{c} \right) \quad (14)$$

Now, picking out these two parts and simplifying, where θ is the angle subtended by $\mathbf{a}$ and $\mathbf{c}$:

$$\begin{aligned}
a^2 c^2 - (\mathbf{a} \cdot \mathbf{c})^2 &= a^2 c^2 - a^2 c^2 \cos^2(\theta) \\
&= a^2 c^2 (1 - \cos^2 \theta) \\
&= a^2 c^2 \sin^2 \theta \\
&= |\mathbf{a} \times \mathbf{c}|^2
\end{aligned}$$

$$\begin{aligned}
a^2 - \mathbf{a} \cdot \mathbf{c} &= \mathbf{a} \cdot \mathbf{a} - \mathbf{a} \cdot \mathbf{c} \\
&= \mathbf{a} \cdot (\mathbf{a} - \mathbf{c}) \\
&= \mathbf{a} \cdot (-\mathbf{b}) \\
&= -\mathbf{a} \cdot \mathbf{b}
\end{aligned}$$

This gives us our final form:

$$\mathbf{B} = \frac{\mu_0 I}{4\pi} \frac{\mathbf{a} \times \mathbf{c}}{|\mathbf{a} \times \mathbf{c}|^2} \left(\frac{\mathbf{a} \cdot \mathbf{c}}{c} - \frac{\mathbf{a} \cdot \mathbf{b}}{b} \right) \quad (15)$$

Now, remembering that this equation is linear and that each coil is made up of straight line filaments we get:

$$\mathbf{B}_{\text{total}} = \sum_{\text{coils}} \sum_{\text{filaments}} \mathbf{B}_{\text{filament}} \quad (16)$$

Now, since we are in free space we use the identity $\mathbf{H} = \frac{\mathbf{B}}{\mu_0}$ [?, p. 285]

0.3 Understanding the forward map.

The problem is framed as an inverse problem. The forward map takes position and orientation data and calculates 8 flux values from each of the coils. Our goal is to approximate an inverse to this map. To do so, we first try to understand the forward map

0.3.1 Definition and properties.

First we define a pose vector $\mathbf{p} \in \mathbb{R}^5$

$$\mathbf{p} = \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix} \quad (17)$$

In actuality, $\mathbf{p}$ is restricted to a set $\Omega \subset \mathbb{R}^5$

$$\Omega = \left\{ \begin{bmatrix} x \\ y \\ z \\ \theta \\ \phi \end{bmatrix} \in \mathbb{R}^5 : \begin{array}{l} x \in (-0.125 \text{ m}, 0.125 \text{ m}) \\ y \in (-0.125 \text{ m}, 0.125 \text{ m}) \\ z \in (0 \text{ m}, 0.25 \text{ m}) \\ \theta \in (0 \text{ rad}, \pi \text{ rad}) \\ \phi \in [0 \text{ rad}, 2\pi \text{ rad}) \end{array} \right\} \quad (18)$$

Then we define a vector of fluxes, $\Phi \in \mathbb{R}^8$

$$\Phi = \begin{bmatrix} \Phi_1 \\ \Phi_2 \\ \Phi_3 \\ \Phi_4 \\ \Phi_5 \\ \Phi_6 \\ \Phi_7 \\ \Phi_8 \end{bmatrix} \quad (19)$$

Now we can write the forward map: $\Phi(\mathbf{p}) : \Omega \subset \mathbb{R}^5 \rightarrow \mathbb{R}^8$. More precisely,

$$\Phi_i = \mathbf{H}_i(x, y, z) \cdot \hat{n}(\theta, \phi) \quad (20)$$

where $\mathbf{H}_i$ is the magnetic field calculated at (x, y, z) due to coil i using equation (15), and $\hat{n}(\theta, \phi)$ is the vector normal to the sensor, defined by:

$$\hat{n}(\theta, \phi) = \begin{bmatrix} \sin \theta \cos \phi \\ \sin \theta \sin \phi \\ \cos \theta \end{bmatrix} \quad (21)$$

Since Φ is smooth, the image of the 5-dimensional domain, Ω , has measure zero in $\mathbb{R}^8$, therefore Φ cannot be surjective.

0.3.2 The Jacobian

Now that we have established that Φ is not surjective we must move our attention to thinking about local invertibility rather than global. To do so we define the Jacobian:

Definition 0.3.1 (Jacobian).

$$J(\mathbf{p}) = \frac{\partial \Phi}{\partial \mathbf{p}} \in \mathbb{R}^{8 \times 5}, J_{i,j} = \frac{\partial \Phi_i}{\partial p_j}$$

Since $\mathbf{H}$ is a finite sum of Biot Savart applications it is a smooth map away from the current filaments. $\hat{n}$ is also a smooth map, therefore Φ will also be smooth away from the current filaments and the Jacobian will be well defined there.

A closed form for $\mathbf{J}$ is cumbersome to find so we instead use finite differences to approximate $\mathbf{J}$

$$\mathbf{J}_{:,i} \approx \frac{\Phi(\mathbf{p} + h\hat{e}_i) - \Phi(\mathbf{p})}{h} \quad (22)$$

where $\hat{e}_i$ is the unit vector in the i th direction, and h is a small real value ($h = 10^{-5}$ in our simulations).

0.3.3 Local Invertibility

The forward map maps $\Omega \subset \mathbb{R}^5$ to $\mathbb{R}^8$, so it is never globally invertible. This raises the question: when, if ever, is the forward map locally invertible.

Definition 0.3.2 (Immersion). An immersion is a smooth map $\phi : P \rightarrow Q$ of manifolds for which $\phi'(p)$ is injective for every $p \in P$

In our case, the derivative of Φ at a point $\mathbf{p}_0 \in \Omega$ is the linear map $J_0 = J(\mathbf{p}_0) \in \mathbb{R}^{8 \times 5}$. Therefore J_0 having full column rank is the condition for Φ to be an immersion at $\mathbf{p}_0$. We can now make some equivalences.

$$J_0^\top J_0 \text{ invertible} \iff J_0 \text{ has full column rank} \iff d\Phi_{\mathbf{p}_0} \text{ injective} \iff \Phi \text{ is an immersion at } \mathbf{p}_0$$

By the rank theorem, for an immersion, every point of Ω lies in an open subset in which the immersion is an embedding.[?]

This means that the pose is unique in a neighbourhood of $\mathbf{p}_0$, distinct nearby poses produce distinct fluxes, and the pose is recoverable from the flux, locally.

Rank deficiency therefore signals regions where the pose cannot be recovered from the flux, even locally. Full column rank is therefore a condition for the inverse problem to be well posed.

0.4 Iterative Methods

Given a function $f : \mathbb{R}^N \rightarrow \mathbb{R}^M$ with $M \geq N$, and a given $y \in \mathbb{R}^M$ suppose we want to find an $x \in \mathbb{R}^N$ that satisfies $y = f(x)$. In the case $m = n$ we could get bijectivity which would allow us to simply invert the map, however, in our case we have $n = 5$ and $m = 8$ so we cannot. In this case we have to use the rank theorem to get a local inverse. This leads us to iterative methods.

0.4.1 Gauss Newton

The Gauss-Newton Algorithm solves this exact kind of problem.

We have a function $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$ and we know $y \in \mathbb{R}^m$, we want to find $x_{\text{true}} \in \mathbb{R}^n$ such that $y = f(x_{\text{true}})$. We start with an initial guess $x_0 \in \mathbb{R}^n$ and define a residual:

$$r(x) = f(x) - y \tag{23}$$

Then we define a cost function:

$$F(x) = \frac{1}{2} \|r(x)\|^2 = \frac{1}{2} r(x)^\top r(x) \tag{24}$$

Suppose now we take a step, $\delta \in \mathbb{R}^N$, we want to find the δ which minimises the cost function.

$$F(x + \delta) = \frac{1}{2} \|r(x + \delta)\|^2 \quad (25)$$

Now, linearising:

$$F(x + \delta) \approx \frac{1}{2} \|r(x) + J(x)\delta\|^2 \quad (26)$$

Since this is a scalar quantity we can take $\nabla_\delta F$ and set it to zero to find the minimum.

$$\nabla_\delta F(x + \delta) = \left(\frac{\partial}{\partial \delta} F(x + \delta) \right)^\top = 0 \quad (27)$$

Now, we consider the k th component of this gradient:

$$(\nabla_\delta F(x + \delta))_k \approx \frac{1}{2} \frac{\partial}{\partial \delta_k} \sum_{j=1}^m \left(r_j(x) + \sum_{i=1}^n J_{ji}(x) \delta_i \right)^2 \quad (28)$$

$$\approx \frac{1}{2} \sum_{j=1}^m 2 \left(r_j(x) + \sum_{i=1}^n J_{ji}(x) \delta_i \right) \frac{\partial}{\partial \delta_k} \left(r_j(x) + \sum_{i=1}^n J_{ji}(x) \delta_i \right) \quad (29)$$

$$\approx \sum_{j=1}^m J_{jk}(x) \left(r_j(x) + \sum_{i=1}^n J_{ji}(x) \delta_i \right) \quad (30)$$

$$\approx \sum_{j=1}^m J_{jk} r_j + \sum_{j=1}^m \sum_{i=1}^n J_{jk} J_{ji} \delta_i \quad (31)$$

$$\approx (J^\top r)_k + (J^\top J \delta)_k \quad (32)$$

Since this applies for all k we get:

$$\nabla_\delta F(x + \delta) \approx J^\top r + J^\top J \delta \quad (33)$$

Setting this equal to zero and solving, yields:

$$\delta = - (J^\top J)^{-1} J^\top r \quad (34)$$

Now that we know the theoretically "best" step to take, we iterate and do this repeatedly:

$$x_{i+1} = x_i - (J(x_i)^\top J(x_i))^{-1} J(x_i)^\top r(x_i) \quad (35)$$

This method has a known failure mode: it has no step-size control. Far from the solution, where the linearisation is poor, the full step can increase the cost and the iteration can diverge.

0.4.2 Gradient Methods

Gauss-Newton fails when it "trusts" the linear model too far from the point at which it was linearised. The opposite approach makes no model of the residual at all. It just moves in the direction that decreases the cost the fastest. Consider the cost function (24) and take its gradient.

$$\nabla_x F(x) = \nabla_x \frac{1}{2} r(x)^\top r(x) \quad (36)$$

Using the same argument as before, taking the k th component of this gradient:

$$(\nabla_x F(x))_k = \frac{1}{2} \frac{\partial}{\partial x_k} \sum_{j=1}^m r_j(x) r_j(x) \quad (37)$$

$$= \frac{1}{2} \sum_{j=1}^m \frac{\partial}{\partial x_k} r_j(x)^2 \quad (38)$$

$$= \frac{1}{2} \sum_{j=1}^m 2r_j(x) \frac{\partial r_j(x)}{\partial x_k} \quad (39)$$

$$= \sum_{j=1}^m r_j(x) \frac{\partial (f_j(x) - y_j)}{\partial x_k} \quad (40)$$

$$= \sum_{j=1}^m r_j(x) \frac{\partial f_j(x)}{\partial x_k} \quad (41)$$

$$= \sum_{j=1}^m r_j(x) J_{jk}(x) \quad (42)$$

$$= (J^\top(x) r(x))_k \quad (43)$$

Gradient descent then steps against this gradient, with the step-size being denoted by the parameter $\eta \in \mathbb{R}$:

$$x_{i+1} = x_i - \eta J(x_i)^\top r(x_i) \quad (44)$$

Because $-\nabla_x F$ is in the direction of steepest local decrease, a sufficiently small η is guaranteed to reduce the cost, so the method is robust far from the solution where Gauss-Newton diverges. Its failure mode is also the opposite of Gauss-Newton's. Near the solution, the gradient becomes small, and since the algorithm has no knowledge of the distance to the solution it bounces back and forth around the minimum.

0.4.3 Levenberg's Contribution

In 1944, Levenberg [?] interpolated between these two algorithms. If we denote each step as δ_{GN} and δ_{GD} , for Gauss Newton and gradient descent, respectively we get that each of these steps solve the equations:

$$(J^\top J)\delta_{GN} = -J^\top r \quad (45)$$

$$\delta_{GD} = -\eta J^\top r \quad (46)$$

Now, letting $\lambda = \frac{1}{\eta}$, multiplying by $I \in \mathbb{R}^{n \times n}$, and interpolating between these two yields:

$$(J^\top J + \lambda I) \delta = -J^\top r \quad (47)$$

Here, λ is known as the damping factor, and another key insight Levenberg had was to vary lambda dynamically. Since GN works better the closer to the solution we get and GD works better the further from the solution we are, the damping is varied dynamically.

We start at step i with $\lambda = \lambda_i$. If $F(x_{i+1}) < F(x_i)$ we are moving in the right direction so can move more into the GN regime, setting $\lambda_{i+1} = 0.1\lambda_i$. However, if $F(x_{i+1}) \geq F(x_i)$ then it is clear that GN is not working and we need to move into a GD dominated regime, thus setting $\lambda_{i+1} = 10\lambda_i$. This ensures that when we are far from the solution we are in a GD dominated regime and as we get closer to the solution GN starts to take over.

0.4.4 The Levenberg Marquardt Algorithm

One downfall of Levenberg's approach is that λ is the same in every dimension, even though the dimensions may operate on different scales. Marquardt [?] solves this by replacing the identity matrix, I with $D = \text{diag}(J^\top J)$ to scale the weights. The Levenberg-Marquardt algorithm solves the following equation at every iteration:

$$(J^\top J + \lambda D) \delta = -J^\top r \quad (48)$$

This means that the algorithm works with different damping in each direction, proportional to the "scale" of that particular direction.

0.4.5 Convergence and the Role of Initialisation

Both Gauss-Newton and Levenberg-Marquardt are local methods, they converge to a stationary point of F , and which stationary point depends on where they start. Since F is typically non-convex it can have multiple local minima. In our problem, in particular, given the high levels of symmetry

[MISSING FIGURE]

figure of local minima

Figure 3: figure of local minima

between the coils, it is highly likely that there are multiple minima. These iterative methods work when the initial guess is in the correct basin of attraction. That then raises the question, how do we get into the right basin of attraction?

0.5 Neural Networks for learning inverses

The iterative methods above invert the forward map each time for every measurement. An alternative would be to spend the computation time upfront and approximate a full inverse map once.

0.5.1 Feed Forward Neural Network

We use a fully-connected feed forward neural network (FFNN) to approximate the inverse. Suppose we have a function $\Phi : \mathbb{R}^N \rightarrow \mathbb{R}^M$ an approximate inverse is defined on the image of Φ , which we call f . We would like to find $y \in \mathbb{R}^N$ such that $y = f(x), x \in \mathbb{R}^M$. To do so, a FFNN with L hidden layers performs the following composition, with $o_0 = x$ and $o_L = \hat{y}$:

$$o_{i+1} = \sigma(W_i o_i + b_i) \quad (49)$$

Where W_i and b_i are the weight matrices and bias vectors (collectively the parameters w), and σ is the nonlinear activation function.

We represent this approximation as $\hat{f}_w : \mathbb{R}^M \rightarrow \mathbb{R}^N$, with $\hat{y} = \hat{f}_w(x)$

The question then becomes how do we find the collection of parameters, w , which does this?

0.5.2 Training a FFNN

Because we possess Φ we can generate unlimited training data, a collection of labeled pairs, (x_i, y_i) . The goal of training is then to minimise the *loss function* with respect to the parameters, w :

Definition 0.5.1 (Loss Function).

$$\hat{L}(w) = \frac{1}{N} \sum_{i=1}^N \left\| \hat{f}_w(x_i) - y_i \right\|^2$$

We minimise the loss using an optimiser, in our case the Adam optimiser [?]. Generalisation is then assessed on a held-out test set never seen during training.